

LECTURE 1 – GAZEBO AND ROS 2

AUTONOMOUS ROBOTIC SYSTEMS ENGINEERING

Topics: Simulator · Physics · Plugins · Fuel · Headless · ROS 2 bridge · RViz · Lab 0.1

TODAY

- **Gazebo** — what it simulates, and how a world is described
- **Connecting it to ROS 2** — the bridge, RViz, simulation time
- **Lab 0.1** — an RGBD camera, from simulation to ROS 2 topics

GAZEBO

WORLDS, PHYSICS AND SENSORS, DESCRIBED IN FILES



WHAT GAZEBO IS

- `gz sim` = a **server** (physics, sensors, plugins) + a **GUI** — two separate processes
- An **entity** is anything that exists in the running world. Every one of these SDF elements becomes one: `<world>` · `<model>` · `<link>` · `<collision>` · `<visual>` · `<joint>` · `<light>` · `<sensor>`
- An entity carries properties — name, pose, mass; **plugins** act on entities every step
- The GUI: play · pause · step · reset · real-time factor · entity tree · plugin menu

A WORLD IS AN SDF FILE

```
<world name="demo">
  <physics name="1ms" type="ignored">
    <max_step_size>0.001</max_step_size>
    <real_time_factor>1.0</real_time_factor>
  </physics>
  <plugin filename="gz-sim-physics-system"
    name="gz::sim::systems::Physics"/>
  <light type="directional" name="sun">...</light>
  <model name="box">
    <pose>0 0 0.5 0 0 0</pose>
    <link name="link">
      <inertial><mass>0.1</mass></inertial>
      <collision name="c">...box...</collision>
      <visual name="v">...box...</visual>
    </link>
  </model>
</world>
```

`<!-- entity: the world itself -->`
`<!-- world settings, not an entity -->`
`<!-- one physics step: 1 ms -->`
`<!-- aim: as fast as reality -->`
`<!-- behaviour, not an entity -->`
`<!-- entity: a light -->`
`<!-- entity: a model, a named object -->`
`<!-- x y z [m], roll pitch yaw [rad] -->`
`<!-- entity: one rigid body -->`
`<!-- what physics needs: mass, inertia -->`
`<!-- entity: the shape that collides -->`
`<!-- entity: the shape you see -->`

 gazebo.org/docs/harmonic/sdf_worlds

EDIT THE FILE, NOT THE WINDOW

- The cycle: edit `world.sdf` → `gz sim -r world.sdf`
- What you change in the GUI lives only in the running simulation — `Reset` clears it, the file is untouched
- A plugin is one `<plugin filename="..." name="...">` line inside `<world>`
- A model from the internet is one `<include>` — later today

WHAT THE PHYSICS COMPUTES

- **mass and inertia** — a 10 kg box is harder to push and to spin than a 0.1 kg one
- **gravity** — 9.81 m/s^2 down: without air drag, our box hits $\sim 310 \text{ m/s}$ after 5000 m
- **forces and torques** — a thruster, the wind, a push applied from the GUI
- **joints** — a wheel's axle (revolute), a drawer (prismatic), a bolted part (fixed)
- **collisions and contacts** — the box stops on the ground instead of falling through it
- **friction** — wheels grip the floor; a caster with low friction slides

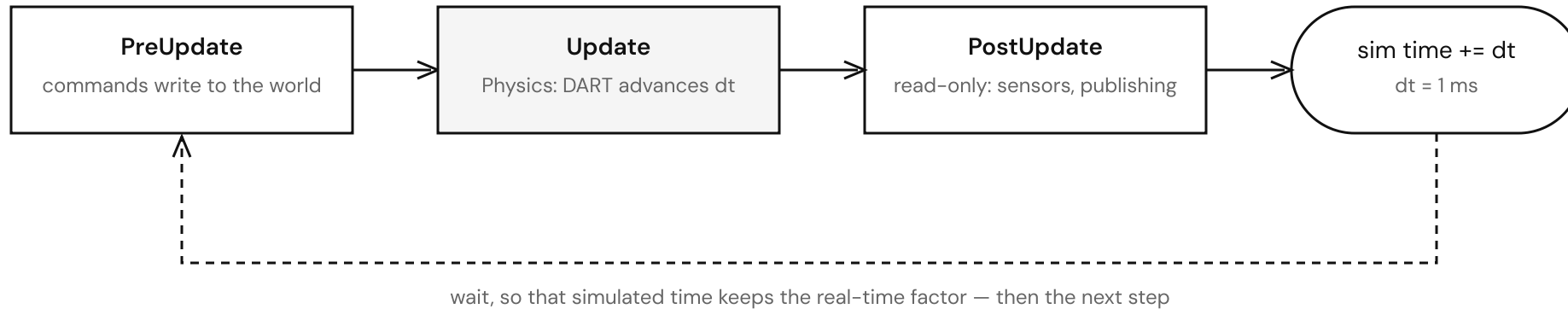
GROUND, GRAVITY, FALLING

- A new world is **empty** — no ground unless you add one
- A model placed in the air **falls** — gravity is on by default: `<gravity>0 0 -9.8</gravity>` in `<world>`
- It **stays** with `<static>true</static>` — a wall, a table, the ground itself
- With no ground it falls **forever** — there is nothing to collide with
- The shortcut: start from Gazebo's `empty.sdf` — it already has a ground and a sun

```
<model name="ground_plane">
  <static>true</static>                                <!-- never moves -->
  <link name="link">
    <collision name="c">                                <!-- what things land on -->
      <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
    </collision>
    <visual name="v">                                   <!-- what you see -->
      <geometry><plane><normal>0 0 1</normal><size>100 100</size></plane></geometry>
    </visual>
  </link>
</model>
```

 sdformat.org/spec?ver=1.11&elem=world

ONE SIMULATION STEP



- $dt = 1\text{ ms}$, so **1 simulated second = 1000 steps**; at real-time factor 1 that is also 1000 steps per second on the wall clock
- Sensors never change the world — they only read what physics just computed

THE DEFAULT ENGINE: DART

- Dynamic Animation and Robotics Toolkit
- Articulated rigid bodies: kinematics, dynamics, collision detection, contact solving
- Gazebo talks to it through the gz-physics library, so the engine is a replaceable plugin
- All of it is open source

 gazebo.org/libs/physics ·  github.com/dartsim/dart (BSD 2-Clause) ·  github.com/gazebo/gz-physics (Apache-2.0) ·  github.com/gazebo/gz-sim (Apache-2.0)

WHAT RELIES ON THE PHYSICS PLUGIN

- **Nothing moves without it.** Checked: with the Physics plugin left out, a box placed at 10 m is still at 10 m
- **Wheeled robots** — wheel–ground contact and friction turn a spinning wheel into motion
- **Drones, planes, boats** — thrust, lift and buoyancy are forces other plugins apply; Physics turns them into motion
- **Arms and grippers** — joint limits, contacts while grasping, an object slipping out
- **Sensors** — an IMU reads the accelerations Physics computed; contact and force–torque sensors read its contacts
- **Testing before hardware** — does it tip over on a slope, collide with the shelf, survive a drop

THREE DEFAULT PLUGINS

PLUGIN	WHAT IT HOLDS
Physics	steps DART, computes motion and contacts
UserCommands	services <code>create</code> · <code>set_pose</code> · <code>remove</code> — how the GUI adds, moves and deletes
SceneBroadcaster	publishes the scene for the GUI — <code>scene/info</code> , <code>pose/info</code> at 60 Hz, <code>state</code> ; without it the window is empty

WHERE THE DEFAULT SET LIVES

```
$ cat /opt/ros/jazzy/opt/gz_sim_vendor/share/gz/gz-sim8/server.config
<plugin entity_name="*" entity_type="world" filename="gz-sim-physics-system" ...>
<plugin entity_name="*" entity_type="world" filename="gz-sim-user-commands-system" ...>
<plugin entity_name="*" entity_type="world" filename="gz-sim-scene-broadcaster-system" ...>
```

- Installed with Gazebo. It is a **list** of plugins, not the plugins themselves
- To change it for yourself: a copy in `~/.gz/sim/8/server.config`, or `GZ_SIM_SERVER_CONFIG_PATH`
- A `<plugin>` list in your SDF **replaces** this set — list every plugin you need

gazebo.org/api/sim/8/server_config.html · github.com/gazebo/gz-sim



SENSOR PLUGINS

<SENSOR TYPE="...">	PLUGIN IN <WORLD>
camera · depth_camera · rgb_camera · thermal_camera · segmentation_camera · boundingbox_camera · wideanglecamera · gpu_lidar	Sensors (gz-sim-sensors-system) + render_engine ogre2 — rendered
imu · magnetometer · altimeter · air_pressure · air_speed · navsat	one plugin each: gz-sim-imu-system, gz-sim-magnetometer-system, ...
contact · force_torque · logical_camera · dvl	gz-sim-contact-system, gz-sim-forcetorque-system, gz-sim-logical-camera-system, gz-sim-dvl-system

- A sensor = its plugin in <world> + a <sensor type="..."> inside a <link>
- Plugin missing → the sensor is silent, and nothing reports an error

 github.com/gazebo/gz-sim/tree/gz-sim8/examples/worlds

OGRE

- An open-source 3D rendering engine; `ogre2` is OGRE-Next, MIT license
- Draws the GUI window **and** every camera image
- **In Lab 0.1 it renders your RGBD camera** — with a window or without one

GETTING A PLUGIN

```
$ gz plugin -i -p /opt/ros/jazzy/opt/gz_sim_vendor/lib/gz-sim-8/plugins/libgz-sim-altimeter-system.so
* Found 1 plugin in library file:
  - gz::sim::v8::systems::Altimeter
* Found 3 interfaces in library file:
  - gz::sim::v8::System
  - gz::sim::v8::ISystemPreUpdate
  - gz::sim::v8::ISystemPostUpdate
```

- A plugin is a library, not a download — the built-in ones ship with Gazebo
- `filename="gz-sim-altimeter-system"` in SDF names this file; the interfaces are the step phases you just saw
- Someone else's plugin: build it, then `export GZ_SIM_SYSTEM_PLUGIN_PATH=<its directory>`
- What is installed: `ls /opt/ros/jazzy/opt/gz_sim_vendor/lib/gz-sim-8/plugins/`
- **Where to look:** every built-in plugin is one folder in `gz-sim/src/systems` — its header lists the SDF parameters; a working world per sensor in `gz-sim/examples/worlds` (previous slide)

 github.com/gazebosim/gz-sim/tree/gz-sim8/src/systems



FUEL

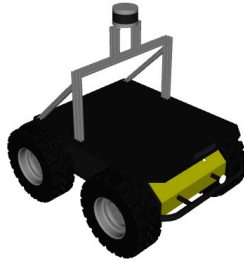
- **Fuel** — an online library of models and worlds for Gazebo
- Referenced by URL; downloaded on first run, cached in `~/.gz/fuel`

```
<include>
  <name>husky</name>
  <pose>0 0 0.2 0 0 0</pose>
  <uri>https://fuel.gazebosim.org/1.0/OpenRobotics/models/COSTAR_HUSKY_SENSOR_CONFIG_1</uri>
</include>
```

 gazebosim.org/docs/harmonic/fuel_insert



REAL MACHINES ON FUEL



Husky, CoSTAR SubT configuration · OpenRobotics, Gazebo Fuel, CC BY 4.0



Boston Dynamics Spot · OpenRobotics, Gazebo Fuel, CC BY 4.0

- Clearpath **Husky**
- Boston Dynamics **Spot**
- PX4 **x500**
- **TurtleBot**
- **X3 UAV** — a generic SubT quadcopter

Not there: DJI Mavic, Crazyflie

WHAT A FUEL MODEL BRINGS

`COSTAR_HUSKY_SENSOR_CONFIG_1` — a Clearpath Husky:

- body — geometry, mass, inertia → **physics works**: it stands, falls, collides
- wheels — 4 links, each attached to `base_link` by a revolute **joint**, the axle
- sensors — RGBD camera, lidars, IMU; silent until their plugins are in `<world>`
- **Kinematics and control are not in the model**: no plugin drives the wheels — it does not move
- ROS integration: **never** included — you bridge it

MAKING THE HUSKY DRIVE

- A **link** is the wheel's body, a **joint** is its axle — **there is no motor entity** in SDF
- **DiffDrive is motor and controller at once**: listens on `/model/husky/cmd_vel`, computes wheel speeds, **sets each wheel joint's velocity directly**
- Inside the `<include>`: `left_joint × 2 · right_joint × 2 · wheel_separation 0.571 m · wheel_radius 0.165 m`
- Velocity set directly = an **ideal motor**: no torque limit, no delay

```
$ gz topic -t /model/husky/cmd_vel -m gz.msgs.Twist -p 'linear: {x: 0.5}'  
$ gz model -m husky --pose # 6 s later  
- Pose [ XYZ (m) ] [ RPY (rad) ]:  
  [3.200720 0.000041 0.132280]
```

WAYS TO MAKE A MODEL MOVE – FOR LATER

WHAT YOU COMMAND	PLUGINS	INPUT
a whole wheeled base	DiffDrive · AckermannSteering (car) · MecanumDrive · TrackedVehicle	a body velocity
one joint at a time	JointController (velocity, or force + PID) · JointPositionController · JointTrajectoryController	a joint velocity, position or trajectory
a raw push	ApplyJointForce · ApplyLinkWrench	a force or torque
the robot, from ROS 2	<code>ros2_control</code> + <code>gz_ros2_control</code> + <code>diff_drive_controller</code>	the same controllers as on a real robot

- The body, wheels and axles always come from the model; motion always comes from one of these
- Next lecture: our own robot model with a plugin from this table · later lectures: `ros2_control`

RUNNING A WORLD

```
$ gz sim -s -v 4 free_fall.sdf
[Msg] Gazebo Sim Server v8.11.0
[Msg] Loading SDF world file[.../worlds/free_fall.sdf].
[Msg] Create service on [/world/free_fall/create_multiple] (async)
[Msg] Remove service on [/world/free_fall/remove] (async)
[Msg] Pose service on [/world/free_fall/set_pose] (async)
```

- `-v 4` — log level: here UserCommands announces the services it just created
- No `-r` → the world waits, paused, for Play or a service call · `-s` server only · `-g` GUI only

 gazebo.org/docs/harmonic/getstarted



LOOKING AT TOPICS

```
$ gz topic -l
/air_pressure
/altimeter
/clock
/world/free_fall/pose/info
/world/free_fall/stats
...
$ gz topic -i -t /air_pressure
Publishers [Address, Message Type]:
  tcp://172.18.0.2:35229, gz.msgs.FluidPressure
No subscribers on topic [/air_pressure]
```

- The same verbs as `ros2 topic` — but these are **Gazebo's** topics; `ros2 topic list` shows none of them
- `gz topic -e -t /altimeter` prints every message as it arrives

WHERE `/altimeter` COMES FROM

1. A sensor on a link — `free_fall.sdf`

```
<world name="free_fall">
  <plugin filename="gz-sim-altimeter-system"
    name="gz::sim::systems::Altimeter"/>
  ...
  <model name="falling_box">
    <link name="link">
      <sensor name="altimeter" type="altimeter">
        <update_rate>30</update_rate>
        <topic>altimeter</topic>  <!-- Gazebo topic -->
      </sensor>
    </link>
  </model>
</world>
```

2. A bridge entry — `free_fall.yaml`

```
- ros_topic_name: "altimeter"
  gz_topic_name: "altimeter"
  ros_type_name: "ros_gz_interfaces/msg/Altimeter"
  gz_type_name: "gz.msgs.Altimeter"
  direction: GZ_TO_ROS
```

3. The chain

- the **Altimeter plugin** reads the link's simulated height every step
- the **sensor** publishes it on Gazebo's `/altimeter`
- the **bridge** republishes it as ROS 2's `/altimeter`
- `/air_pressure` is the same, with `air_pressure` and **AirPressure**

LOOKING AT MODELS

```
$ gz model --list
Available models:
  - ground
  - falling_box
$ gz model -m falling_box --pose          # a few seconds after Play
Model: [8]
  - Name: falling_box
  - Pose [ XYZ (m) ] [ RPY (rad) ]:
    [0.000000 0.000000 4785.940000]
    [0.000000 -0.000000 0.000000]
```

- **A model** is one named object: links + joints, and sometimes sensors and plugins
- **Where models come from:** a `<model>` written in the world file · an `<include>` from Fuel or a local folder · spawned at run time (a GUI shape button, the `create` service)
- Here both `ground` and `falling_box` are written in `free_fall.sdf`
- `gz model` reads the running world directly — no bridge, no ROS; `--link` / `--joint` list the parts: next lecture

 gazebo-sim.org/api/sim/8/model_command.html



HEADLESS

```
$ gz sim -s -r free_fall.sdf &           # -s: server only, no window
$ gz topic -l                             # use it through topics...
/air_pressure
/altimeter
/clock
...
$ gz model --list                         # ...and through the world state
  - ground
  - falling_box
$ gz sim -g                               # later, on a machine with a display: attach a window
```

- Physics and sensors run exactly as with a window — the window is only a client
- Everything you would click is also a topic or a service
- Where it pays: containers, remote machines, CI, long runs

 gazebo.org/docs/harmonic/getstarted

CAMERAS WITHOUT A DISPLAY

- The camera image is rendered **by Gazebo**, with OGRE, where the server runs — RViz only draws the finished image it receives
- So rendering is needed even with no Gazebo window and no RViz at all
- No display on that machine (server, container, SSH) → OGRE renders off-screen, through EGL
- `--headless-rendering` asks for that up front. Without it OGRE first looks for a display, then falls back:

```
$ gz sim -s -r rgbd.sdf  
[Wrn] [Ogre2RenderEngine.cc:551] Unable to open display: . Trying to run in headless mode.
```

- Checked in our container, software OpenGL, no GPU: the RGBD camera streams ~30 Hz with the flag and without it

WEB VISUALIZATION

- **Why:** the server has no window — a container, a remote machine — and you still want to see the scene
- **What you see:** the 3D scene — models and their moving poses; not the full GUI
- Checked: the server in our container answers on port 9002 — `HTTP/1.1 101 Switching Protocols`
- **How:** `gz sim -s -r world.sdf` → `gz launch -v 4 websocket.gzlaunch` → app.gazebosim.org/visualization → Connect
- Needs the websocket server from `gz-launch` — a separate Gazebo component, installed on its own

GAZEBO ↔ ROS 2

ONLY WHAT YOU BRIDGE EXISTS IN ROS



THE BRIDGE

- Gazebo has its own transport; ROS 2 sees **only the topics you bridge** — nothing is bridged by default
- **One** `parameter_bridge` process carries any number of topics — one entry per topic
- `/topic@ros_type[ gz_type` — `[ Gazebo → ROS · ]` `ROS → Gazebo · @` both ways
- Frame names are translated on the way: `falling_box::link::altimeter` in Gazebo arrives as `falling_box/link/altimeter`

```
$ ros2 run ros_gz_bridge parameter_bridge '/rgbd_camera/image@sensor_msgs/msg/Image[ gz.msgs.Image'
```

ONE SENSOR, FOUR TOPICS – THE BRIDGE FILE

```
# Gazebo -> ROS 2: one entry per topic
- ros_topic_name: "rgb_camera/image"
  gz_topic_name: "rgb_camera/image"
  ros_type_name: "sensor_msgs/msg/Image"
  gz_type_name: "gz.msgs.Image"
  direction: GZ_TO_ROS      # colour picture

- ros_topic_name: "rgb_camera/depth_image"
  gz_topic_name: "rgb_camera/depth_image"
  ros_type_name: "sensor_msgs/msg/Image"
  gz_type_name: "gz.msgs.Image"
  direction: GZ_TO_ROS      # metres per pixel
```

```
- ros_topic_name: "rgb_camera/points"
  gz_topic_name: "rgb_camera/points"
  ros_type_name: "sensor_msgs/msg/PointCloud2"
  gz_type_name: "gz.msgs.PointCloudPacked"
  direction: GZ_TO_ROS      # depth as 3D points

- ros_topic_name: "rgb_camera/camera_info"
  gz_topic_name: "rgb_camera/camera_info"
  ros_type_name: "sensor_msgs/msg/CameraInfo"
  gz_type_name: "gz.msgs.CameraInfo"
  direction: GZ_TO_ROS      # intrinsics: depth -> 3D
```

`ros2 run ros_gz_bridge parameter_bridge --ros-args -p config_file:=rgb.yaml` — all four in one process; bridge only
what you consume

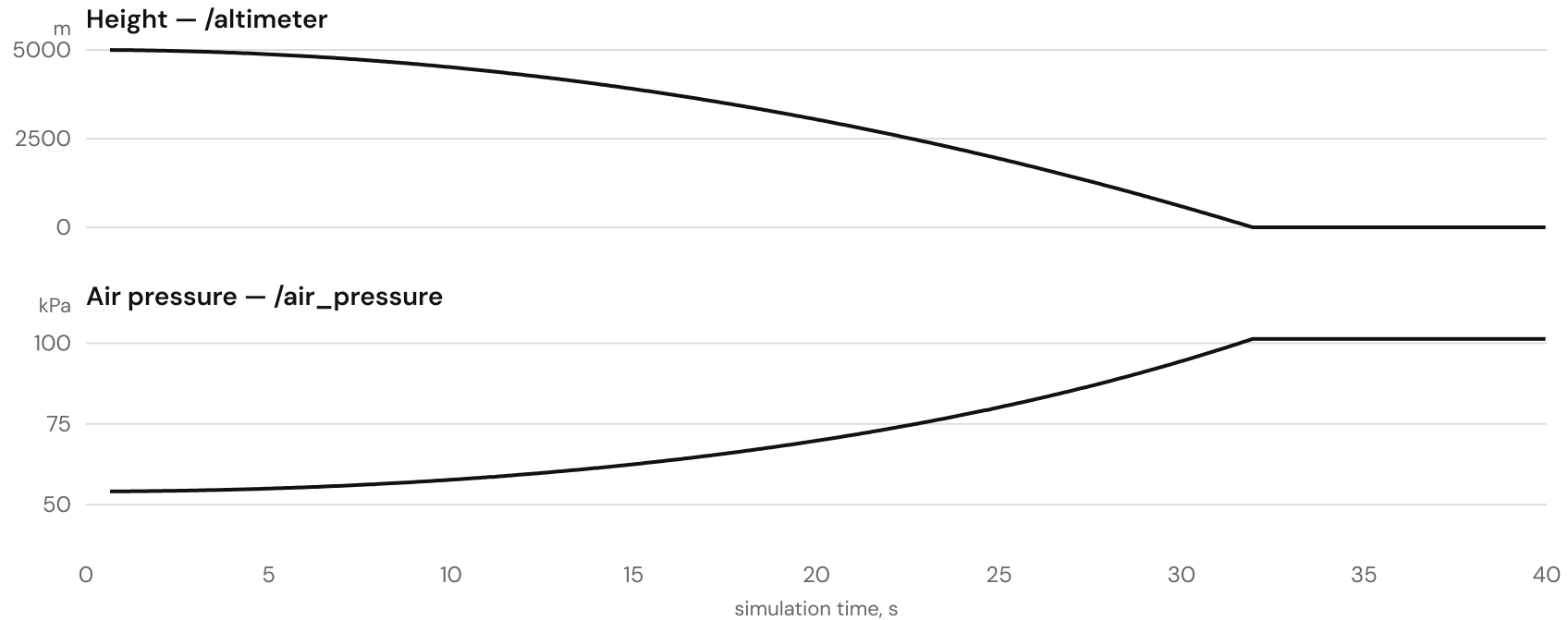
github.com/gazebosim/ros_gz/blob/jazzy/ros_gz_bridge/README.md · github.com/gazebosim/ros_gz/blob/jazzy/ros_gz_sim_demos/README.md

IN ROS 2, WHILE IT FALLS

```
$ ros2 topic echo /altimeter --once
header:
  stamp: {sec: 13, nanosec: 497000000}
  frame_id: falling_box/link/altimeter
vertical_position: -892.69          # metres below the start
vertical_velocity: -132.27         # m/s, and growing
vertical_reference: 5000.0
$ ros2 topic echo /air_pressure --field fluid_pressure --once
61701.83                          # Pa – it was ~54 000 at 5000 m
```

- **Nothing here is measured** — Gazebo computes both values from the box's simulated position, every step
- `/altimeter` ← `<sensor type="altimeter">` on the box's link + the **Altimeter** plugin in `<world>`: height relative to where the box started
- `/air_pressure` ← `<sensor type="air_pressure">` + the **AirPressure** plugin: standard-atmosphere pressure at that height
- Gazebo topics `gz.msgs.Altimeter`, `gz.msgs.FluidPressure` → the bridge → ROS 2 `ros_gz_interfaces/msg/Altimeter`, `sensor_msgs/msg/FluidPressure`
- The stamp is **simulation** time, not the clock on the wall

THE FALL, PLOTTED



- Recorded from `/altimeter` and `/air_pressure` in ROS 2, during one run of `free_fall.sdf`
- Lands at **~31.9 s** of simulated time — exactly $\sqrt{2 \cdot 5000 / 9.81} \approx 31.9$ s; pressure **54.1 → 101.3 kPa**
- Both curves come from one simulated position — the sensors only translate it

RVIZ – SEEING WHAT ROS SEES

- A ROS 2 viewer: draws whatever is on ROS topics — it simulates nothing, and it is not part of Gazebo
- **Sensors:** camera images, depth, point clouds, laser scans, range cones
- **Robots:** the robot model, TF frames, joint efforts, odometry
- **Navigation:** maps and costmap cells, planned paths, poses; tools to set a pose estimate and send a goal
- **Anything else:** markers — shapes your own node draws
- **Lab 0.1:** Image on `/rgb_camera/image` · PointCloud2 on `/rgb_camera/points` · Fixed Frame `rgb_camera/link/rgb_camera`

WHY SIMULATION TIME

- A simulation runs slower or faster than reality, pauses, steps — the wall clock knows none of this
- `use_sim_time: true` → a node's time comes from `/clock`, published by **Gazebo**
- `use_sim_time: false` — the default → the computer's **system clock**

 design.ros2.org/articles/clock_and_time.html



LAB 0.1 – WHAT YOU DO

A sensor in a simple world → its data in ROS 2. You choose the sensor. Not graded.

	A · RGBD CAMERA	B · BAROMETER + ALTIMETER
world	Gazebo's <code>sensors_demo.sdf</code> (level 0) · your <code>rgbd.sdf</code> (level 1)	Gazebo's <code>sensors.sdf</code> (level 0) · your <code>pressure.sdf</code> — a box dropped from 1000 m (level 1)
plugins	defaults + Sensors with <code>ogre2</code> — rendered	defaults + AirPressure + Altimeter
Gazebo topics	4 × <code>/rgbd_camera/...</code>	<code>/air_pressure</code> , <code>/altimeter</code>
bridge	4 entries — <code>rgbd.yaml</code>	2 entries — <code>pressure.yaml</code>
check	<code>ros2 topic hz /rgbd_camera/image</code> → ~30 Hz	<code>ros2 topic echo /altimeter --field vertical_position</code> falls to -1000
see it	RViz: Image + PointCloud2, Fixed Frame <code>rgbd_camera/link/rgbd_camera</code>	the listener's log: height, speed, pressure
your code	<code>rgbd_listener</code> — image processing	<code>pressure_listener</code> — e.g. height from pressure

Steps: world → run it (`-s` without a display) → `gz topic -l` → bridge → check in ROS 2 → look at it

▶ gazebo-sim.org/docs/harmonic/sensors · gazebo-sim.org/docs/harmonic/ros2_launch_gazebo

LAB 0.1 – FILES, LEVEL 1

```
gz_rgb_lab/      # option A; gz_pressure_lab/ has exactly the same shape
├─ package.xml   # dependencies: rclpy, sensor_msgs, ros_gz_sim, ros_gz_bridge
├─ setup.py      # installs worlds/, config/, launch/; makes the listener runnable
├─ setup.cfg     # where `ros2 run` finds the node – generated
├─ resource/gz_rgb_lab # empty marker file – generated
├─ launch/rgb.launch.py # Gazebo + bridge + listener in one command; gui:=true for a window
├─ worlds/rgb.sdf # plugins, ground, light, a box, a sphere, the RGBD camera
├─ config/bridge.yaml # the bridge, 4 entries (option B: 2)
└─ gz_rgb_lab/
    ├─ __init__.py # empty
    └─ rgb_listener.py # image + depth; TODO: your processing
```

```
$ colcon build --packages-select gz_rgb_lab && source install/setup.bash
$ ros2 launch gz_rgb_lab rgb.launch.py # option B: gz_pressure_lab pressure.launch.py
```

Level 0 needs none of these files — only the world that ships with Gazebo and commands.

NEXT LAB – WHAT YOU DO

A model with sensors from Fuel, in your own world, one of its sensors in ROS 2. **Graded.**

1. **Pick a model with sensors** — e.g. `COSTAR_HUSKY_SENSOR_CONFIG_1`: an RGBD camera, an IMU, lidars
2. **Write your own world** — `<include>` the model by its Fuel URL; list the plugins: the three defaults + **Sensors** (`ogre2`) for its cameras + **Imu** for its IMU
3. **Run it** — with a window or headless; the first run downloads the model, needs the network
4. **Find its sensors** — `gz topic -l`; a sensor without its own `<topic>` publishes under its full path:
`/world/<world>/model/husky/link/base_link/sensor/camera_front/image` · `gz.msgs.Image`
`/world/<world>/model/husky/link/base_link/sensor/imu_sensor/imu` · `gz.msgs.IMU`
5. **Bridge one sensor into ROS 2** — a YAML entry from that long Gazebo name to a short ROS name
6. **Show it** — `ros2 topic hz`, and RViz with the display that matches the type
7. **README** — how to run it from zero, which topic, what you saw

 gazebo.org/docs/harmonic/ros_gz_project_template_guide



NEXT LAB – FILES

```
my_fuel_lab/
├─ package.xml          # dependencies: ros_gz_sim, ros_gz_bridge (+ rviz2)
├─ setup.py             # installs worlds/, config/, launch/ (+ rviz/)
├─ setup.cfg            # where ros2 finds things – generated
├─ resource/my_fuel_lab # empty marker file – generated
├─ worlds/fuel_world.sdf # plugins (defaults + Sensors + the sensor's own), ground, light, <include> of the model
├─ config/bridge.yaml   # one entry: the model's full Gazebo topic -> a short ROS 2 topic
├─ launch/fuel.launch.py # Gazebo + the bridge (+ rviz2 with your layout), one command
├─ rviz/fuel.rviz       # optional: a saved RViz layout with the right display
└─ README.md            # run from zero · which sensor and topic · what you saw
```

- No Python node is required — configuration only
- The model comes from Fuel at run time and is cached in `~/.gz/fuel`; nothing of it goes into your repository
- Check before you bridge: `gz topic -i -t <topic>` shows the Gazebo type the YAML entry needs

WHAT'S NEXT

Lecture 2 — The Robot Model

The robot model · plugins that move it · SDF and URDF

 gazebo.org/docs/harmonic

